CS471 Project Proposal: Hardware support for Mixed-Format Quantization

Mihai Indreias Filip Corlan
EPFL, Lausanne, Switzerland

Abstract— Quantization in the context of LLMs reduces weight storage, but packed representations introduce overhead from unpacking, scaling, and offset correction during inference. We plan to investigate whether Avant-Garde's hardware support for scaled formats provides similar performance benefits for a modern 27B-parameter Qwen GGUF model (3.8 Fast UD-Q4_K_M). We will profile `llama.cpp` on an NVIDIA H100, identify the main bottlenecks, and evaluate format-aware microarchitectural extensions in Accel-Sim.

I. Introduction

Low-bit quantization reduces model size and memory traffic, making LLMs easier to deploy. A storage format is not a native arithmetic format, so the original weights must be reconstructed. In `llama.cpp`, GGUF tensors can use different encodings within one model; their kernels must bridge packed weights and the arithmetic supported by the GPU. Optimized software can fuse conversion with computation, so conversion overhead must be measured within kernels.

Avant-Garde [1] introduces an Operand Transformer block into the pipeline and modified Tensor Cores for scaled numeric formats. It evaluates the HBFP, MX9, and MXFP8 formats on ViT, BERT, and GPT-2. We will investigate how Avant-Garde's proposed changes apply to GGUF encodings with floating-point scales, nested integer scales, and affine offsets. Supporting these encodings may require extensions beyond the original scaling datapath.

Our target is the `Qwen3.8-27B-UD-Q4_K_M.gguf` GGUF checkpoint [2].

The three main questions we plan to answer are:

1. What limits prefill and decoding on the H100?
2. How much overhead is attributable to handling quantized representations?
3. Which hardware changes reduce this overhead without unacceptable numerical error?

Our hypothesis is that linear-layer kernels account for more than 30% of GPU execution time in selected workloads. We will test this by profiling inference on a real GPU and, if the hypothesis holds, evaluate whether Avant-Garde can accelerate the relevant numeric formats.

II. Design Space

A. Formats and computation

For a linear layer, $Y = XW + b$, where $X \in \mathbb{R}^{M\times K}$ and $W \in \mathbb{R}^{K\times N}$, the arithmetic work is approximately $2MKN$ operations. Prefill processes multiple tokens at once, whereas single-sequence decoding typically has $M = 1$. Representative workloads must therefore include both matrix–matrix and matrix–vector products, which have different data-reuse and memory-bandwidth characteristics.

Table 1 summarizes the formats found in the selected checkpoint [3]. Here q denotes an unpacked integer code, d a block scale, s_j a local scale, m_j a local minimum code, and $\hat{w}$ a reconstructed weight.

Table 1: Weight reconstruction for the selected GGUF formats.

Format	Block size	Reconstruction
F32	1	$\hat{w} = w_{\text{stored}}$
Q8_0	32	$\hat{w} = dq$
Q3_K	256	$\hat{w} = ds_j q$
Q4_K	256	$\hat{w} = ds_j q - d_{\min} m_j$
Q5_K	256	$\hat{w} = ds_j q - d_{\min} m_j$
Q6_K	256	$\hat{w} = ds_j q$
IQ4_NL	32	$\hat{w} = dL[q]$
IQ4_XS	256	$\hat{w} = ds_j L[q]$
IQ3_S	256	$\hat{w}_i = d(1 + 2s_j)\sigma_i G[k]_r$

Q8_0 uses signed 8-bit codes. Q5_K contains eight 32-weight subblocks with packed scale and minimum codes; Q6_K contains sixteen 16-weight subblocks with signed scales and centered 6-bit codes. The block scales use FP16 representations.

Q4_K uses unsigned 4-bit codes in eight 32-weight subblocks. Its scale and minimum codes are packed unsigned 6-bit values, with FP16 global scales d and $d_{\min}$. Q3_K uses sixteen 16-weight subblocks: $q \in [-4, 3]$ is reconstructed from two low bits and a high-bit mask, and $s_j \in [-32, 31]$ is obtained by unpacking a 6-bit scale code and subtracting 32.

IQ4_NL uses 4-bit indices into the fixed nonlinear lookup table L, rather than uniformly spaced integer val-

ues:

$$L = (-127, -104, -83, -65, -49, -35, -22, -10, 1, 13, 25, 38, 53, 69, 89, 113).$$

IQ4_XS uses the same table in eight 32-weight subblocks per 256-weight block. Its signed local scale s_j is a packed 6-bit code minus 32.

IQ3_S uses eight 32-weight subblocks per 256-weight block. Each 9-bit index k selects four magnitudes from the fixed 512-entry grid $G = \texttt{iq3s_grid}$, and $r \in \{0, 1, 2, 3\}$ selects one component. Separate sign bits give $\sigma_i \in \{-1, +1\}$, while the unsigned 4-bit local scale code s_j produces the multiplier $1 + 2s_j$. These reconstruction rules follow the `llama.cpp` dequantization routines [4].

B. Candidate hardware support

We will identify representative, optimized `llama.cpp` kernels and construct comparable versions that use Avant-Garde's proposed operand-transformation mechanisms where applicable. Our primary comparison will be between software-only handling of quantized weights and the existing Avant-Garde architecture, rather than assuming that a new hardware extension is required. We will investigate additional hardware changes only where profiling and simulation reveal a specific gap in the existing design.

As a stretch goal, we will investigate lookup-table-based IQ formats, drawing on approaches such as FLUTE and LUT Tensor Core [5, 6].

III. Evaluation

A. Hardware profiling and benchmark selection

We will run the selected Qwen model with `llama.cpp` on an NVIDIA H100 and use NVIDIA profiling tools to measure where GPU time is spent under different inference conditions. We will distinguish prefill from decoding and record kernel execution time, throughput, achieved memory bandwidth, Tensor Core utilization, instruction mix, and register usage.

We plan to test prompt lengths of 128, 512, and 2048 tokens and batch sizes of 1, 8, and 32, generating 128 tokens per run. The profile will account for linear layers, attention, normalization, data movement, and any recurrent or state-related operations present in the model. We will then select kernels that preserve the observed matrix shapes, quantization formats, activation types, and dispatch paths. This benchmark suite will represent the dominant workloads without requiring full-model traces in the simulator.

B. Simulation, baselines, and correctness

We will first establish a working Accel-Sim configuration with Hopper support [7, 8] and validate representative baseline kernels against H100 measurements. We will compare execution cycles and memory behavior and report any significant discrepancies between simulation and hardware.

Next, we will compare software-only kernels against kernels using Avant-Garde's hardware mechanisms [1]. Initially, we will focus on relevant formats such as Q4_K and Q5_K. Comparisons will use representative input shapes and equivalent simulated GPU resources. Any added hardware behavior will be modeled with explicit operations, data dependencies, memory accesses, and timing rather than by simply removing conversion instructions from an execution trace.

We will check kernel outputs against a reference that dequantizes the weights before multiplication, reporting absolute and relative error. Performance evaluation will cover per-kernel cycles and speedups, with attention to matrix shape, batch size, transformation overhead, and memory bandwidth. Finally, we will combine the simulated kernel speedups with the measured inference profile to estimate the potential overall speedup, accounting for operations that remain unaffected. We will distinguish these estimates from throughput measured on the H100.

IV. Milestones

1. We will establish a working simulation environment by verifying the existing Avant-Garde and Accel-Sim implementations, validating the baseline configuration, and confirming access to H100 profiling.
2. We will profile prefill and decoding for the selected Qwen checkpoint on an H100, identify the operations that dominate GPU execution time, and construct a representative benchmark suite covering the relevant kernels and quantization formats.
3. We will compare software-only quantization handling with Avant-Garde's hardware mechanisms, initially focusing on Q4_K and Q5_K. We will validate numerical correctness and evaluate performance across representative matrix dimensions and batch sizes.
4. We will report per-kernel speedups and estimate their effect on overall inference performance using the measured workload profile. If time permits, we will investigate lookup-table-based IQ formats and possible extensions to Avant-Garde's architecture.

References

[1] M. Gil et al., "Avant-Garde: Empowering GPUs with Scaled Numeric Formats," *ISCA*, 2025. doi:10.1145/3695053.3731100.

[2] Unsloth, "Qwen3.8-27B-GGUF," model repository. Hugging Face repository.

[3] GGML contributors, "`llama.cpp`: Quantization Block Definitions," `ggml/src/ggml-common.h`. GitHub repository.

[4] GGML contributors, "`llama.cpp`: Dequantization Routines," `ggml/src/ggml-quants.c`. GitHub source.

[5] H. Guo et al., "Fast Matrix Multiplications for Lookup Table-Quantized LLMs," *Findings of EMNLP*, 2024. doi:10.18653/v1/2024.findings-emnlp.724.

[6] Z. Mo et al., "LUT Tensor Core: A Software-Hardware Co-Design for LUT-Based Low-Bit LLM Inference," *ISCA*, 2025. doi:10.1145/3695053.3731057.

[7] M. Khairy et al., "Accel-Sim: An Extensible Simulation Framework for Validated GPU Modeling," *ISCA*, 2020. doi:10.1109/ISCA45697.2020.00047.

[8] J. Pan et al., "Architecting the Next Generation of Asynchronous, Distributed GPUs for the AI Era," 2026. arXiv:2608.22602.